



Banco de Dados

Prof. Elson de Abreu



Trigger

Trigger

- Uma trigger é um tipo especial de stored procedure. É associada a uma tabela e não pode ser chamada diretamente.
 - Uma trigger é executada automaticamente, após um comando de INSERT, UPDATE ou DELETE na tabela a qual a trigger esteja associada (Triggers DML exceto “instead of”).
 - Normalmente as triggers são usadas para garantir uma integridade de CHECK mais complexa ou alterações em cascata em tabelas, dentre outras tarefas. Também não é incomum ver bancos de dados com triggers garantindo a integridade referencial.
 - A partir da versão 2005 do SQL Server, as triggers são divididas em Triggers de DML e Triggers de DDL.
 - Sequência de execução Trigger DML (exceto triggers “instead of” – não iremos abordar):
 1. As constraints são verificadas.
 2. A operação (INSERT, UPDATE, DELETE) é executada.
 3. A trigger é disparada.
- ✓ *As tabelas podem conter várias triggers para uma mesma ação. Uma trigger faz parte de uma transação. Os gatilhos DML AFTER podem ser definidos apenas em tabelas.*

Trigger DML

```
CREATE TRIGGER [ schema_name . ]trigger_name ON { table | view } [ WITH  
    <dml_trigger_option> [ ,...n ] ]  
{ FOR | AFTER | INSTEAD OF } { [ INSERT ] [ , ] [ UPDATE ] [ , ] [ DELETE ] } [ WITH APPEND ] [ NOT FOR  
    REPLICATION ]  
AS { sql_statement [ ; ] [ ...n ] | EXTERNAL NAME <method specifier [ ; ] > }
```

Observações:

on [owner.]table_name → determina a qual tabela a trigger está associada.

insert, update, delete → determina em qual operação a trigger será executada.

AFTER é o padrão se apenas a palavra-chave FOR for especificada.

- *Dica: A partir de agora, utilize o comando SET NOCOUNT OFF para desativar as mensagens do tipo "1 row(s) affected", sempre que alguma linha for atualizada, faça isso no início do seu código e ao término SET NOCOUNT ON para reativar as mensagens. Porém @@ROWCOUNT só é atualizado quando estiver em SET NOCOUNT ON.*

Trigger DDL

```
CREATE TRIGGER trigger_name ON { ALL SERVER | DATABASE } [ WITH  
    <ddl_trigger_option> [ ,...n ] ]  
{ FOR | AFTER } { event_type | event_group } [ ,...n ]  
AS { sql_statement [ ; ] [ ...n ] | EXTERNAL NAME < method specifier > [ ; ] }
```

Trigger DML

- Existem 2 tabelas especiais, utilizadas em instruções de triggers:

✓ **INSERTED**: registra os dados após alteração.

✓ **DELETED**: registra os dados antes da alteração.

- O SQL Server automaticamente cria e gerencia essas tabelas temporárias (criadas em memória), utilizadas para testar os efeitos das alterações realizadas.

Exemplo 1 DML

```
IF OBJECT_ID ('tr_EMP') IS NOT NULL
```

```
BEGIN
```

```
    DROP TRIGGER tr_EMP
```

```
END
```

```
GO
```

```
CREATE TRIGGER tr_EMP ON emp
```

```
FOR INSERT, UPDATE, DELETE AS
```

```
    PRINT 'Inserted:'
```

```
    SELECT empno, nome, salario, deptno FROM inserted
```

```
    PRINT 'Deleted:'
```

```
    SELECT empno, nome, salario, deptno FROM deleted
```

```
GO
```

Obs.: A trigger tr_EMP irá exibir o registro antes e após os dados serem inseridos, alterados ou excluídos, nos dados da tabela “Emp” (banco de dados “Empresa”).

Exemplo 1 *continuação*

- Observe a inserção de um novo funcionário:

`insert into emp (empno, nome, salario, deptno) values (9000, 'Elson', 5000, 10)`

```
Inserted:
empno      nome      salario      deptno
-----
9000      Elson      5000,00      10

(1 row(s) affected)

Deleted:
empno      nome      salario      deptno
-----

(0 row(s) affected)

(1 row(s) affected)
```


Exemplo 1 *continuação*

- Reajustando o salário dos funcionários do departamento de código 10:

`update emp set salario = salario * 1.5 where deptno = 10`

✓ Observe os registros da tabela *DELETED* e compare-os com os da tabela *INSERTED*.

✓ Os registros de *DELETED* são os “antigos”, ou seja, antes da atualização.

Inserted:			
empno	nome	salario	deptno

9000	Elson	7500,00	10
8000	Stephen	3000,00	10
7934	MILLER	1950,00	10
7839	KING	7500,00	10
7782	CLARK	3675,00	10

(5 row(s) affected)

Deleted:			
empno	nome	salario	deptno

9000	Elson	5000,00	10
8000	Stephen	2000,00	10
7934	MILLER	1300,00	10
7839	KING	5000,00	10
7782	CLARK	2450,00	10

(5 row(s) affected)

Exemplo 1 *continuação*

- Excluindo registros:

`delete from emp where deptno = 10`

```
Inserted:
empno      nome                salario                deptno
-----
(0 row(s) affected)

Deleted:
empno      nome                salario                deptno
-----
9000      Elson                7500,00                10
8000      Stephen             3000,00                10
7934      MILLER              1950,00                10
7839      KING                7500,00                10
7782      CLARK               3675,00                10

(5 row(s) affected)

(5 row(s) affected)
```

Exemplo 2 (DML)

```
CREATE TRIGGER tr_EMPSALARIO ON EMP
AFTER UPDATE AS

    DECLARE @NEW_SAL money, @OLD_SAL money

    IF UPDATE(SALARIO) -- testando se a coluna "salario" está sendo modificada
    BEGIN

        SELECT @NEW_SAL = salario
        FROM inserted

        SELECT @OLD_SAL = salario
        FROM deleted

        IF @NEW_SAL < @OLD_SAL
        BEGIN
            RAISERROR('Não é possível diminuir o salário do empregado',16,1)
            ROLLBACK
        END
    END
```

Obs: esta trigger explica bem o que a frase “Uma trigger faz parte de uma transação” quer dizer. Veja o comando rollback!

Exemplo 2 *DML*

```
CREATE TRIGGER tr_EMPSALARIO ON EMP
AFTER UPDATE AS
    DECLARE @NEW_SAL money, @OLD_SAL money
    IF UPDATE(SALARIO) -- testando se a coluna "salario" está sendo modificada
    BEGIN
        SELECT @NEW_SAL = salario
        FROM inserted

        SELECT @OLD_SAL = salario
        FROM deleted

        IF @NEW_SAL < @OLD_SAL
        BEGIN
            RAISERROR('Não é possível diminuir o salário do empregado',16,1)
            ROLLBACK
        END
    END
END
```

Obs.: esta trigger explica bem o que a frase “Uma trigger faz parte de uma transação” quer dizer. Veja o comando rollback!

Exemplo 2 *continuação*

- Ao tentar reduzir o salário do funcionário, veja o que acontece:

```
update emp set salario = 1000 where empno = 7902
```

- Observe a mensagem *“The Transaction ended in the trigger. The batch has been aborted.”*, a trigger abortou a transação, mesmo não tendo emitido um *“begin tran”* (explícito). Lembre-se das transações implícitas ?

```
Msg 50000, Level 16, State 1, Procedure tr_EMPSALARIO, Line 14
Não é possível diminuir o salário do empregado
Msg 3609, Level 16, State 1, Line 1
The transaction ended in the trigger. The batch has been aborted.
```

Exemplo 3 DDL

```
CREATE DATABASE EXEMPLO_TRIGGER
```

```
GO
```

```
USE EXEMPLO_TRIGGER
```

```
GO
```

```
CREATE TABLE LOG
```

```
(  
  MAQUINA SYSNAME DEFAULT HOST_NAME(),  
  DADOS XML  
)
```

```
GO
```

```
CREATE TRIGGER TR_SEGURANCA_TABELAS
```

```
ON DATABASE
```

```
FOR CREATE_TABLE, ALTER_TABLE, DROP_TABLE
```

```
AS
```

```
INSERT INTO LOG(DADOS) VALUES (EVENTDATA())
```

```
GO
```

```
CREATE TABLE PESSOA
```

```
( COD INT IDENTITY, NOME VARCHAR(50))
```

```
GO
```

```
SELECT * FROM LOG
```

```
GO
```

```
ALTER TABLE PESSOA ADD PRIMARY KEY ( COD )
```

```
GO
```

```
SELECT * FROM LOG
```

```
GO
```

```
DROP TABLE PESSOA
```

```
GO
```

```
SELECT * FROM LOG
```

```
GO
```

- Neste exemplo criamos uma tabela de log para as operações de CREATE_TABLE, ALTER_TABLE, DROP_TABLE no banco de dados.

Exemplo 4 DDL

```
USE EMPRESA
```

```
GO
```

```
IF EXISTS (SELECT * FROM sys.triggers
```

```
    WHERE name = 'TR_SEGURANCA')
```

```
DROP TRIGGER TR_SEGURANCA ON DATABASE
```

```
GO
```

```
CREATE TRIGGER TR_SEGURANCA
```

```
ON DATABASE FOR DROP_TABLE
```

```
AS
```

```
ROLLBACK -- O ROLLBACK CANCELA A EXCLUSÃO DA TABELA
```

```
RAISERROR('DESABILITE A TRIGGER DE SEGURANCA ANTES!',16,1) -- GERANDO UMA  
EXCEÇÃO
```

```
GO
```

```
DROP TABLE SALGRADE -- 1# TESTANDO A TRIGGER
```

```
GO
```

```
DISABLE TRIGGER TR_SEGURANCA ON DATABASE -- DESABILITANDO A TRIGGER
```

```
GO
```

```
DROP TABLE SALGRADE -- 2# TESTANDO A TRIGGER
```

```
GO
```

```
ENABLE TRIGGER TR_SEGURANCA ON DATABASE -- HABILITANDO A TRIGGER
```

```
GO
```

```
DROP TABLE EMP -- 3# TESTANDO A TRIGGER
```

```
GO
```

- Neste exemplo criamos uma trigger de segurança, para evitarmos exclusão de tabelas em nosso banco de dados, seja acidental ou não.

Atividade 1

- Utilizando o bd “Empresa”, crie uma trigger na tabela **EMP**, de maneira que não seja possível trocar o departamento ao qual o empregado está vinculado para um departamento inativo, e que também não permita inserir um novo empregado vinculado a um departamento inativo.
- ✓ Dica: verifique se a coluna com o código do departamento está sendo realmente alterada.
- Obs.: altere a tabela DEPT, criando uma nova coluna chamada INATIVO, do tipo BIT.

Atividade 2

- Utilizando o bd “Locadora”, crie uma trigger na tabela **LOCACAO**, de maneira que, ao registrar uma locação, verifique se a fita a ser alugada esteja disponível (status ‘D’). Caso esteja, altere o seu status para alugada (status ‘L’). Caso contrário, gere uma exceção.